CS 회고

Fetch · REST API · Promise · async/await · BOM · DOM

1. Fetch와 REST API

웹 애플리케이션의 기본 구조

1. **브라우저(클라이언트)** : 화면 구성
2. **REST API 서버** : 요청 처리, 업무 로직 수행, DB 접근
3. **데이터베이스**

HTTP 통신 요청/응답 과정

- 클라이언트 `Fetch(HTTPRequest)` → REST API 서버 ↔ 데이터베이스(조회 결과)
  - HTTPRequest로 **URL, HTTP 메서드(POST 등), 본문**을 전달
- 브라우저 `[HTML5 + JS + DOM + Event]` ← `JSON(HTTPResponse)` (요청 처리 · 업무 로직)
  - HTTPResponse로 **상태 정보, 헤더, 응답 본문**을 반환

Fetch API : JS로 HTTP 통신을 수행하는, 요청과 응답을 다루는 도구

- 별도 설치 없이 브라우저(또는 최신 Node.js)에서 바로 사용 가능
- GET(조회), POST(생성), PUT/PATCH(수정), DELETE(삭제) 등 다양한 메서드 지원

구분	역할	주요 구성 요소
클라이언트	사용자 화면을 표시하고 서버에 요청을 보낸다.	브라우저, HTML, JavaScript, DOM
REST API 서버	요청을 해석하고 업무 로직 및 데이터 조회를 수행한다.	URL, HTTP 메서드, 응답 상태
데이터베이스	애플리케이션 데이터를 저장하고 조회한다.	회원, 게시글, 상품 등의 자원
Fetch API	JavaScript에서 HTTP 통신을 수행하는 브라우저 제공 API. 서버 응답을 비동기적으로 받아 처리한다.	`fetch()`, Promise, Response

REST API 통신에서는 요청 시 `fetch()`를 사용하고, 응답은 주로 JSON으로 반환받는다.

```
fetch("https://api.example.com/users/1")
  .then((response) => response.json())
  .then((user) => console.log(user));
```

`response.json()` : 응답 본문을 JS가 처리할 수 있도록 JSON 형식으로 변환한다.

- 단일 자원 → 객체
- 여러 자원 → 객체 배열

2. RESTful URL

> REST API는 자원을 URL로 표현하고, 수행할 작업은 HTTP 메서드로 구분하여 설계한다.

HTTP 요청 시 URL, 메서드, 헤더, 본문으로 서버에 작업을 요청한다. 이때 URL과 메서드에 따라 원하는 작업이 결정된다.

연습용 가짜 REST API 서비스: https://jsonplaceholder.typicode.com

목적	HTTP 메서드	URL 예시	설명
목록 조회	GET	`/users`	모든 회원 목록을 조회한다.
단건 조회	GET	`/users/1`	식별자가 1인 회원을 조회한다.
생성	POST	`/users`	새로운 회원 데이터를 생성한다.
전체 수정	PUT	`/users/1`	기존 자원 전체를 새 데이터로 교체한다.
부분 수정	PATCH	`/users/1`	변경할 속성만 부분적으로 수정한다.
삭제	DELETE	`/users/1`	식별자가 1인 자원을 삭제한다.

POST, PUT, PATCH(서버에 데이터를 전달하는 요청)는 본문과 헤더를 따로 설정하고, `Content-Type` **헤더를 명시**해야 한다.

```
fetch("https://api.example.com/posts", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    title: "새 게시글",
    body: "본문 내용",
    userId: 1
  })
});
```

3. Promise와 비동기 응답 처리

네트워크 통신은 서버로부터 응답이 오기까지 시간이 걸리므로 **비동기 작업**으로 처리된다. 응답이 오기 전까지 `fetch()`의 반환 결과는 Promise 객체이다.

Promise : 비동기 통신의 완료/실패 등 결과를 반환하는 객체

Promise의 상태

상태	의미	처리 방식
`pending`	서버 응답 또는 비동기 작업을 기다리는 상태	결과값을 즉시 사용하면 안 된다.
`fulfilled`	작업이 정상 완료되어 결과값을 사용할 수 있는 상태	`then()` 또는 `await` 이후에 값을 처리한다.
`rejected`	네트워크 오류 또는 예외로 작업이 실패한 상태	`catch()` 또는 `try...catch`로 처리한다.

`response.json()` 역시 Promise를 반환하므로 바로 출력하지 않고, `then()`을 통해 비동기 작업의 결과를 다음 콜백으로 전달한다.

```
fetch("https://jsonplaceholder.typicode.com/users/3")
  .then((response) => response.json()) // Response 객체를 받아 JSON으로 변환
  .then((user) => {                    // 전달받은 JSON 데이터를 출력
    console.log(user.name);
    console.log(user.email);
  })
  .catch((error) => {
    console.error("통신 오류:", error);
  });
```

위 예시는 3번 유저 한 명(단일 자원, 객체)을 가져오지만, 아래처럼 `/users`로 여러 유저(여러 자원, 객체 배열)를 가져오는 경우에는 `.find()`로 원하는 조건의 객체를 선택할 수 있다.

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then((response) => response.json())
  .then((users) => {
    const user = users.find((item) => item.id === 3);
    console.log(user.name, user.email);
  });
```

4. async/await

> Promise의 결과를 기다리면서 비동기 코드를 작성하는 키워드. HTTP 상태 검증과 `try...catch` 예외 처리가 필요하다.

- `await` : Promise가 해결될 때까지 `async` 함수 **내부의** 다음 처리를 대기시킨다. (**브라우저 전체를 멈추는 것이 아니다!**)
- 사용하려면 함수 앞에 `async`를 명시해야 한다.

```
async function loadUserPosts() { // async 명시
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users/1/posts"
    );

    if (!response.ok) { // HTTP 오류 명시적 처리
      throw new Error(`HTTP 오류: ${response.status}`);
    }

    const posts = await response.json();
    console.log(posts);
    console.log(posts[0].title);
  } catch (error) {
    console.error("요청 처리 실패:", error);
  }
}
```

`!response.ok`로 200번대 성공 응답인지 확인하고, 아닐 경우 `response.status`로 구체적인 상태 코드를 확인한다.

Promise.all

여러 비동기 작업을 **동시에 시작**하고 모두 끝나길 기다린다. 모든 요청이 완료된 시점에 결과를 함께 처리한다. → 순서대로 `await` 하는 것보다 훨씬 빠르게 여러 요청을 처리할 수 있다.

```
const [usersResponse, postsResponse] = await Promise.all([
  fetch("https://jsonplaceholder.typicode.com/users"),
  fetch("https://jsonplaceholder.typicode.com/posts")
]);
```

비동기 처리 방식 2가지 비교

```
// case 1 : .then() 사용
fetch('https://jsonplaceholder.typicode.com/posts/1')
  .then(response => response.json())
  .then(json => console.log(json)); // then()으로 기다리므로 정상 출력

// case 2 : async/await
async function getData() {          // async 명시 함수 선언
  const response = await fetch('https://jsonplaceholder.typicode.com/posts/1');
  const json = await response.json();
  console.log(json);                // await으로 기다리므로 정상 출력
}
getData();
```

5. window 객체와 BOM

`window` : 브라우저 환경의 최상위 전역 객체. DOM 문서와 브라우저 관련 객체를 관리한다.

객체	역할	대표 기능
`window`	브라우저 창의 최상위 객체	전역 함수, 팝업, 이벤트 관리
`document`	현재 HTML 문서를 객체로 표현	DOM 선택, 생성, 수정, 삭제
`location`	현재 URL과 페이지 이동 정보 관리	URL 조회, 페이지 이동, 새로고침
`history`	브라우저 방문 이력 관리	이전 페이지, 다음 페이지 이동
`navigator`	브라우저와 실행 환경 정보 제공	언어, 사용자 에이전트, 위치 정보
`screen`	사용자 화면 및 모니터 정보 제공	화면 크기, 사용 가능 영역

→ 이러한 브라우저 관련 객체의 집합을 **BOM**(Browser Object Model)이라고 한다.

6. DOM 트리

> DOM은 HTML 문서를 트리 구조의 객체로 표현한다.

HTML 문서는 최상위 요소에서 하위 요소로 이어지는 트리 형태이다. JS는 이 트리를 이용해 원하는 요소(Element)를 선택하고, 속성·내용·스타일을 변경할 수 있다.

예를 들어 `getElementById()`는 고유 id 값을 기준으로 하나의 요소를 선택한다. *(id는 문서 안에서 유일하다!)*

```
const title = document.getElementById("title");

title.style.color = "skyblue";
console.log(title.innerHTML);
```

선택 메서드	선택 기준	반환 형태
`getElementById()`	id 값	단일 Element
`getElementsByClassName()`	클래스 이름	HTMLCollection
`getElementsByTagName()`	태그 이름	HTMLCollection
`querySelector()`	CSS 선택자	첫 번째 일치 Element
`querySelectorAll()`	CSS 선택자	NodeList

- `querySelector` : CSS 선택자로 단일 element를 가져온다.
- `querySelectorAll` : CSS 선택자로 element를 모두 가져온다. → NodeList 형태로 반환
- 둘 다 CSS 선택자 문법을 그대로 사용할 수 있다.

```
const boxes = document.querySelectorAll(".box");

for (const box of boxes) {
  box.style.border = "3px solid green";
}
```

HTMLCollection과 NodeList가 반환되는 경우, 일반적으로 순회에 `for...of` 를 사용한다.

Element와 Node

- `Element` : HTML 태그 요소
- `Node` : 요소뿐 아니라 텍스트, 공백, 주석까지 포함하는 더 넓은 개념

DOM 탐색

부모, 자식, 형제 관계를 이용하여 원하는 요소를 수정하거나 추가·삭제할 수 있다.

- `parentElement` : 부모 element 반환
- `children` : 자식 element 반환
- `childNodes` : 텍스트 노드, 주석 노드를 포함한 모든 노드 반환

```
const first = document.getElementById("first");
const parent = first.parentElement;

parent.style.border = "1px solid red";
console.log(parent.children);
```